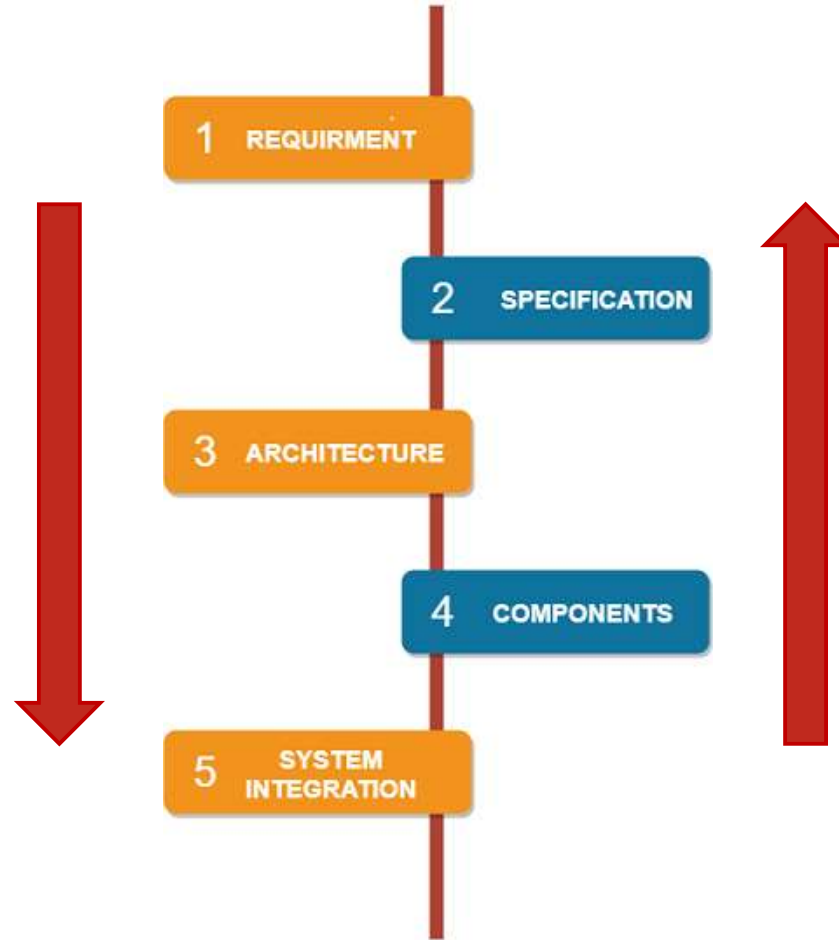


EMBEDDED SYSTEM DESIGN PROCESS

EMBEDDED SYSTEM DESIGN PROCESS

Top-down - From description of the system to concrete details.



bottom-up - Start with components to build a system.

Ref. "Computers as components: principles of embedded computing system design" by Wayne Wolf

EMBEDDED SYSTEM DESIGN PROCESS

- ❑ **The major goals of the design to be considered are :**
 - **Manufacturing cost**
 - **Performance**
 - **Power consumption**
- ❑ **Tasks which need to be performed at each step,**
 - **analyze the design at each step - meet specification**
 - **refine the design - add details**
 - **verify the design - system goals**

EMBEDDED SYSTEM DESIGN PROCESS

REQUIREMENTS

- ❑ Informal descriptions gathered from the customer are known as requirements
- ❑ Requirements are refined into a specification to begin the designing of the system architecture
- ❑ Can be functional or non-functional requirements
 - **Functional:** Needed output as a function of input
 - **Non-functional:** performance, cost, physical size, weight, and power consumption

EMBEDDED SYSTEM DESIGN PROCESS

REQUIREMENTS – EXAMPLE (GPS Navigation System)

Product Name	GPS Navigation System
Purpose	Consumer grade moving map for driving use
Inputs	Power button two control button
Outputs	Back-lit LCD Display 400x600
Functions	Uses 5-receiver GPS system; three user-selectable resolutions; always displays current latitude and longitude
Performance	Updates screen within 0.25 seconds upon movement
Power	100mW
Cost	30 USD
Size And Weight	2" x 6" and 250 grams

EMBEDDED SYSTEM DESIGN PROCESS

SPECIFICATION

- ❑ Requirements gathered is refined into a specification.
- ❑ Specification serves as the contract between the customers and the architects.
- ❑ Specification is essential to create working systems with a minimum of designer effort.
- ❑ Specific, understandable and accurately reflect the customer's requirements.

EMBEDDED SYSTEM DESIGN PROCESS

SPECIFICATION - EXAMPLE (GPS Navigation System)

- ❑ **Specification would include details for several components:**
 - **Data received from the GPS satellite**
 - **Map data**
 - **User interface**
 - **Operations that must be performed to satisfy customer requests**
 - **Background actions**

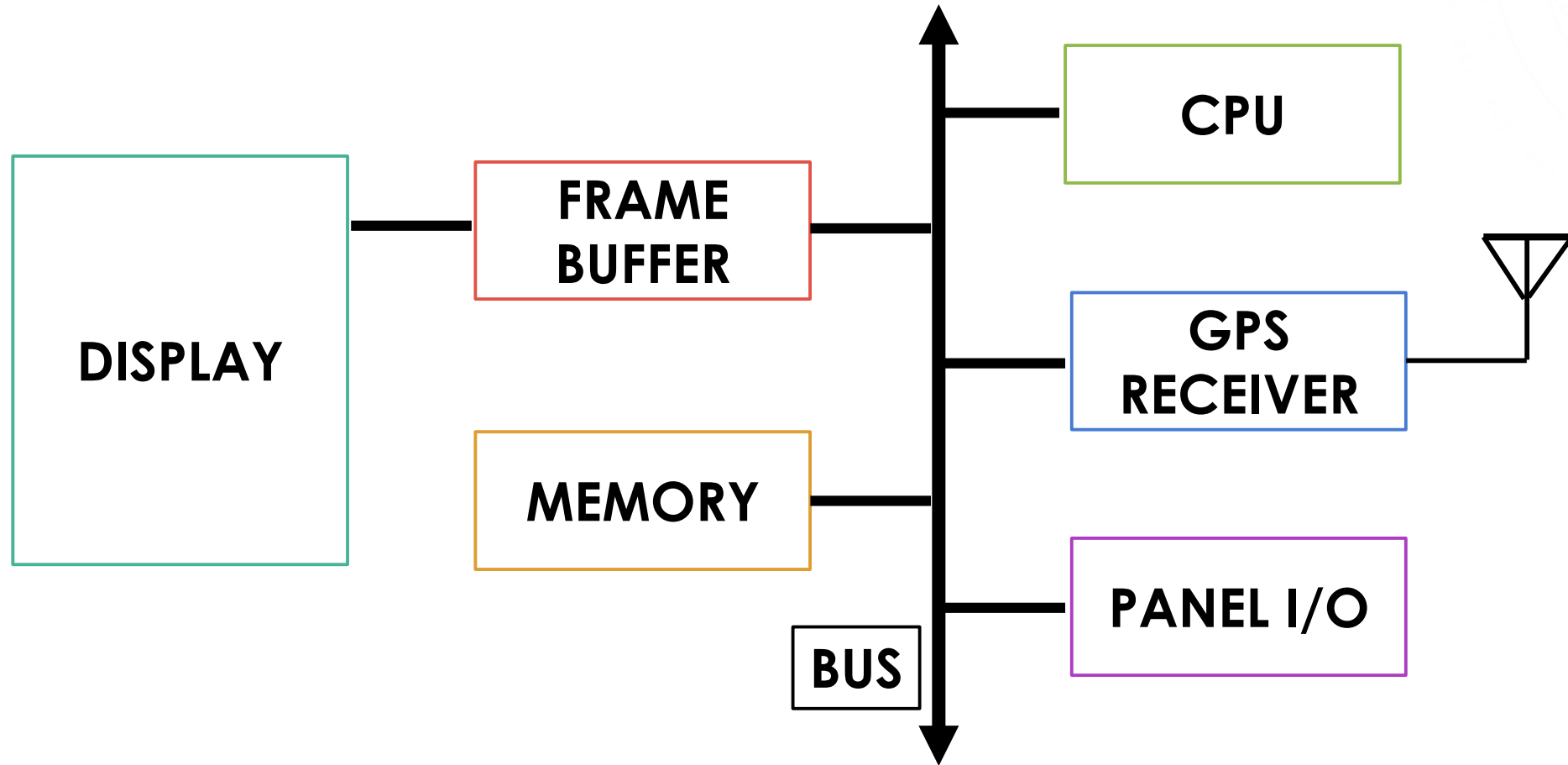
EMBEDDED SYSTEM DESIGN PROCESS

ARCHITECTURE

- ❑ The specification describes only the functions of the system
- ❑ Implementation of the system is described by the Architecture
- ❑ The architecture is a plan for the overall structure of the system.
- ❑ It will be used later to design the components
- ❑ The system block diagram may be refined into two block diagrams - hardware and software

EMBEDDED SYSTEM DESIGN PROCESS

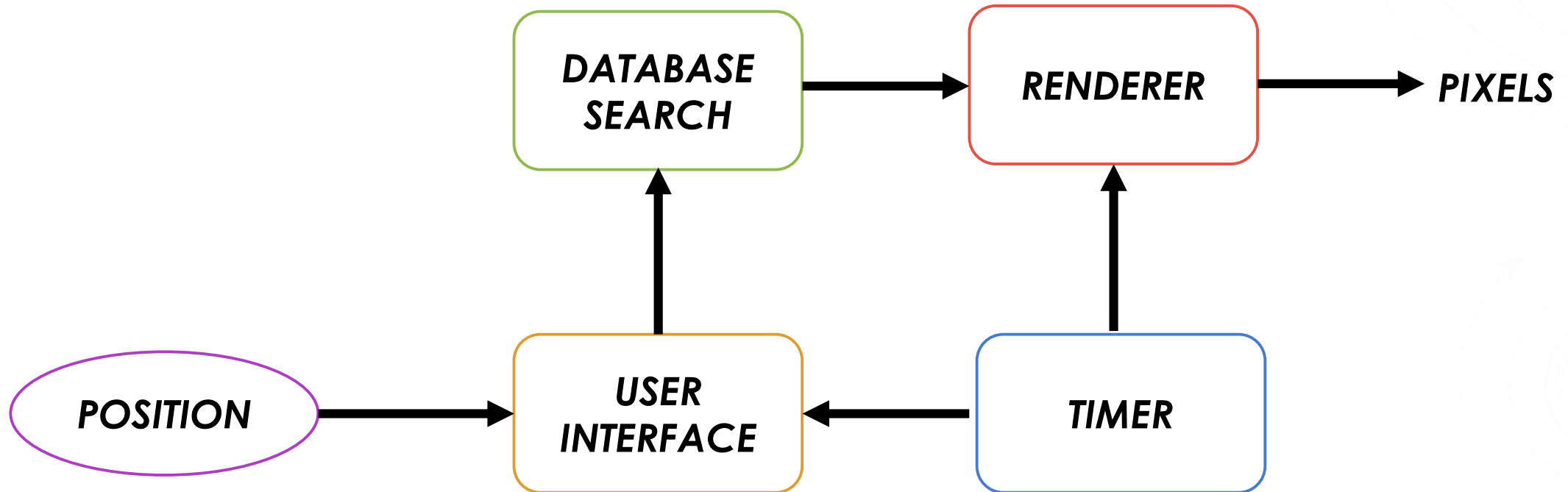
ARCHITECTURE - EXAMPLE (GPS Navigation System)



SYSTEM DESIGN - HARDWARE

EMBEDDED SYSTEM DESIGN PROCESS

ARCHITECTURE - EXAMPLE (GPS Navigation System)



SYSTEM DESIGN - SOFTWARE

EMBEDDED SYSTEM DESIGN PROCESS

HARDWARE AND SOFTWARE COMPONENTS

- ❑ The architectural description specify what components we need
- ❑ The component design effort builds those components in conformance to the architecture and specification
- ❑ Includes both hardware and software modules
- ❑ Some of the components will be ready-made (example :CPU, memory chips)

EMBEDDED SYSTEM DESIGN PROCESS

HARDWARE AND SOFTWARE COMPONENTS - EXAMPLE

- ❑ In the moving map, **GPS receiver** is a **predesigned** standard hardware component.
- ❑ **Topographic software** is a **standard software** module which uses standard routines to access the database.
- ❑ **Printed circuit board** are the components which needs to be designed.
- ❑ When creating these embedded software modules, **ensure the system runs properly** in real time and that it does not take up more memory space than allowed.

EMBEDDED SYSTEM DESIGN PROCESS

SYSTEM INTEGRATION

- ❑ After the components are built, they are integrated
- ❑ Bugs are typically found during the system integration
- ❑ By debugging a few modules at a time, simple bugs can be discovered
- ❑ By fixing the simple bugs early, more complex bugs can be discovered
- ❑ Appropriate debugging facilities during design which can help to ease system integration problems.

EMBEDDED SYSTEM DESIGN PROCESS – EXAMPLE

AUTOMATIC CHOCOLATE VENDING MACHINE

REQUIREMENTS

☐ Purpose:

- To sell chocolate through an ACVM from which children can automatically purchase the chocolates.
- The payment is by inserting the coins into a coin-slot.

☐ Inputs:

- Coins of different denominations through a coin slot.
- User commands.

☐ Outputs:

- Chocolate and signal to the system that subtracts the cost from the value of amount collected.
- Display of the menus for GUIs, time and date, advertisements, welcome and thank messages.

AUTOMATIC CHOCOLATE VENDING MACHINE

REQUIREMENTS

❑ Functions of the system:

- A child sends commands to the system using a GUI. It must consists of the LCD display and keypad units.
- The child inserts the coins for cost of chocolate and the machine delivers the chocolate.
- If the coins are not inserted as per the cost of chocolate in reasonable times then all coins are refunded. If the coin amount more, the excess amount is refunded along with chocolate.
- The coins for the chocolates purchased collect inside the machine in a collector channel, so that owner can get the money, again through appropriate commands using the GUI.
- USB wireless modem enables communication through Internet to the ACVM owner.

AUTOMATIC CHOCOLATE VENDING MACHINE

REQUIREMENTS

❑ Design metrics:

- **Power Dissipation:** As required by mechanical units, display units and computer system
- **Performance:** One chocolate in two minutes and 256 chocolates before next filling of chocolates into the machine.[Assumed]
- **Process Deadlines:** Machine waits for maximum 30 s for the coins and machine should deliver the chocolate within 60 s.
- **User Interfaces:** Graphic at LCD or touch screen display on LCD and commands by children or machine owner through fingers on keypad or touch screen
- **Engineering Cost:** US\$ 50000 (assumed)
- **Manufacturing Cost:** US\$ 1000 (assumed)

AUTOMATIC CHOCOLATE VENDING MACHINE

SPECIFICATIONS

- **Alphanumeric keypad** on the top of the machine. A child interaction with it when buying a chocolate. Owner commands and interaction with the machine.
- **Three line LCD display unit** on the top of the machine. Displays menus, entered text, pictograms, and welcome, thank and other messages, and time and date. Child as well as the ACVM owner GUIs with the machine using keypad and display.

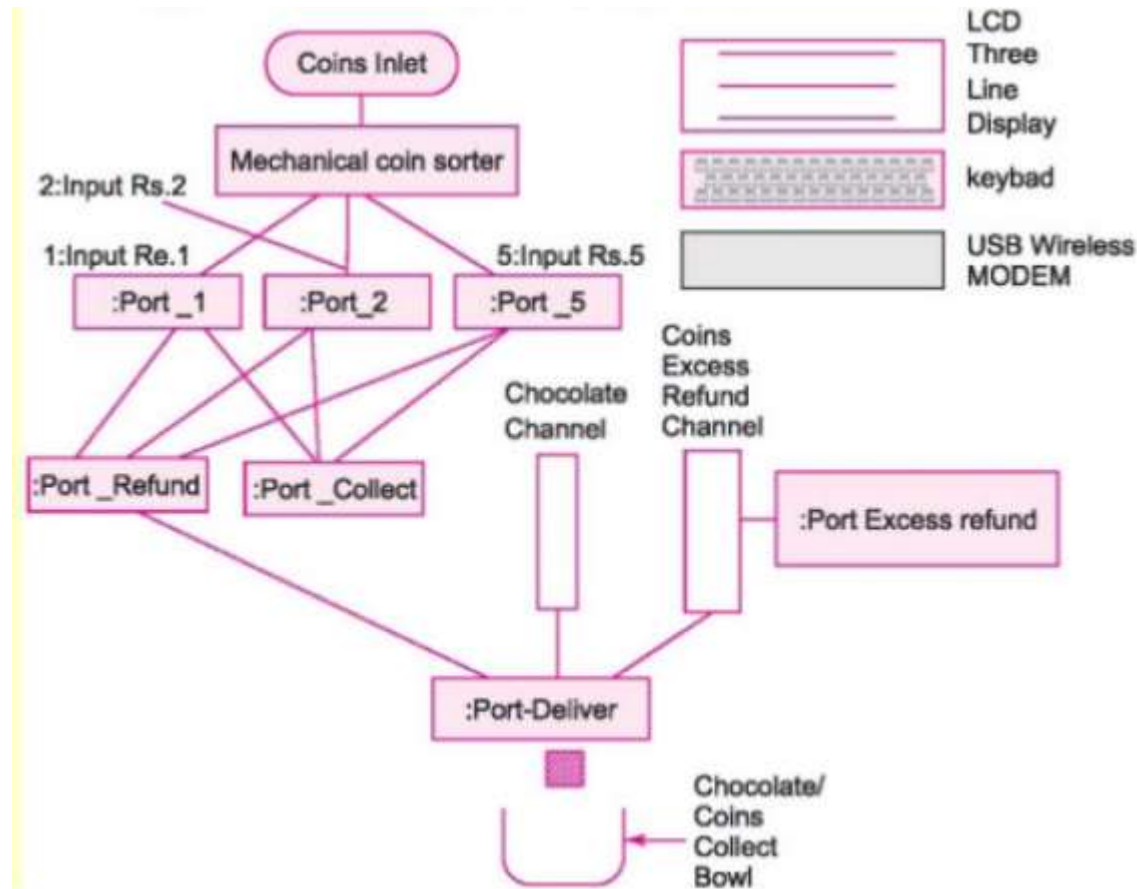
AUTOMATIC CHOCOLATE VENDING MACHINE

SPECIFICATIONS

- Coin insertion slot so that the child can insert the coins to buy a chocolate.
- Delivery slot to collect the chocolate, and coins if refunded.
- Internet connection port so that owner can interact with ACVM from remote.

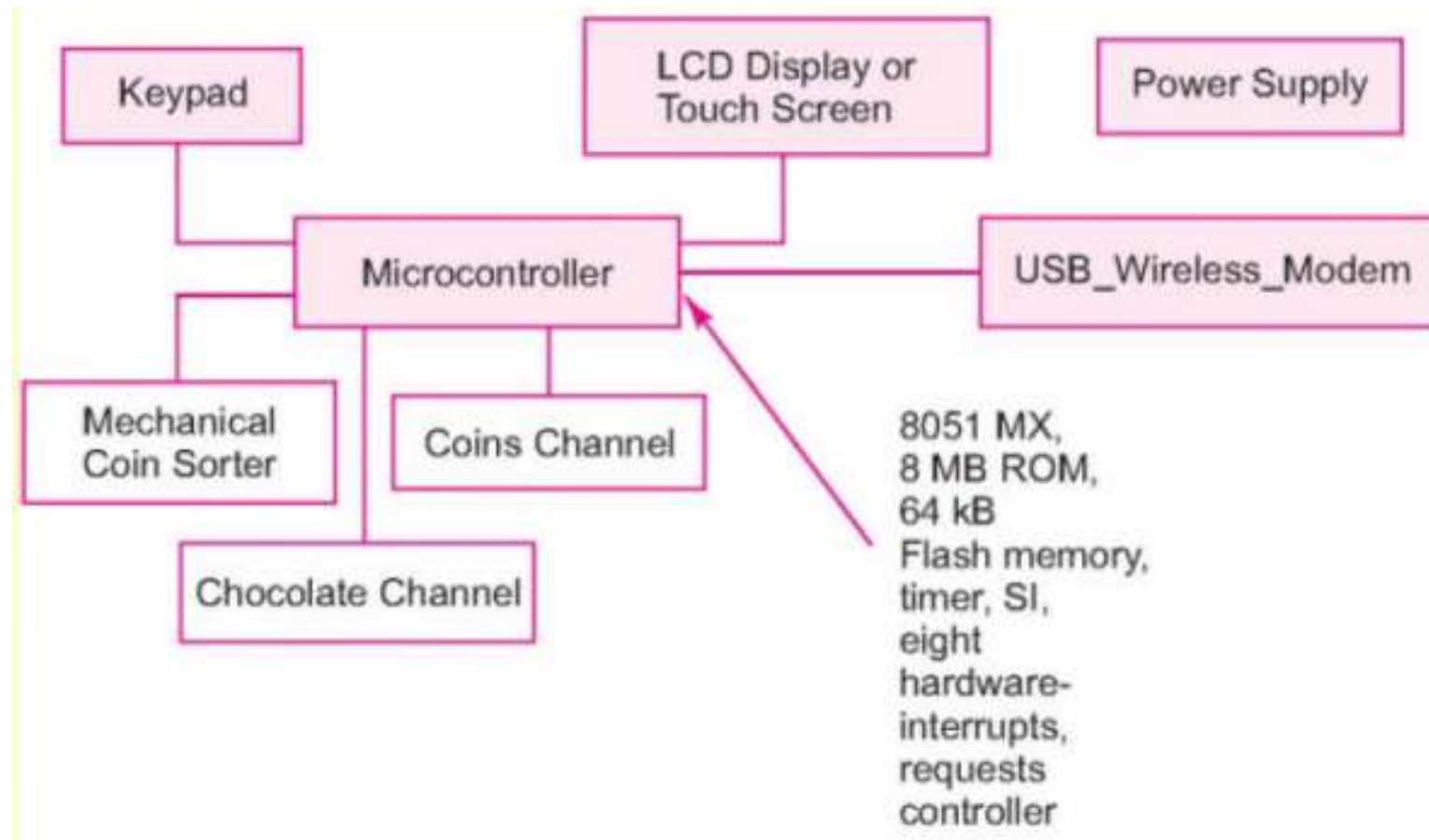
AUTOMATIC CHOCOLATE VENDING MACHINE

ARCHITECTURE – OVERALL SYSTEM DIAGRAM



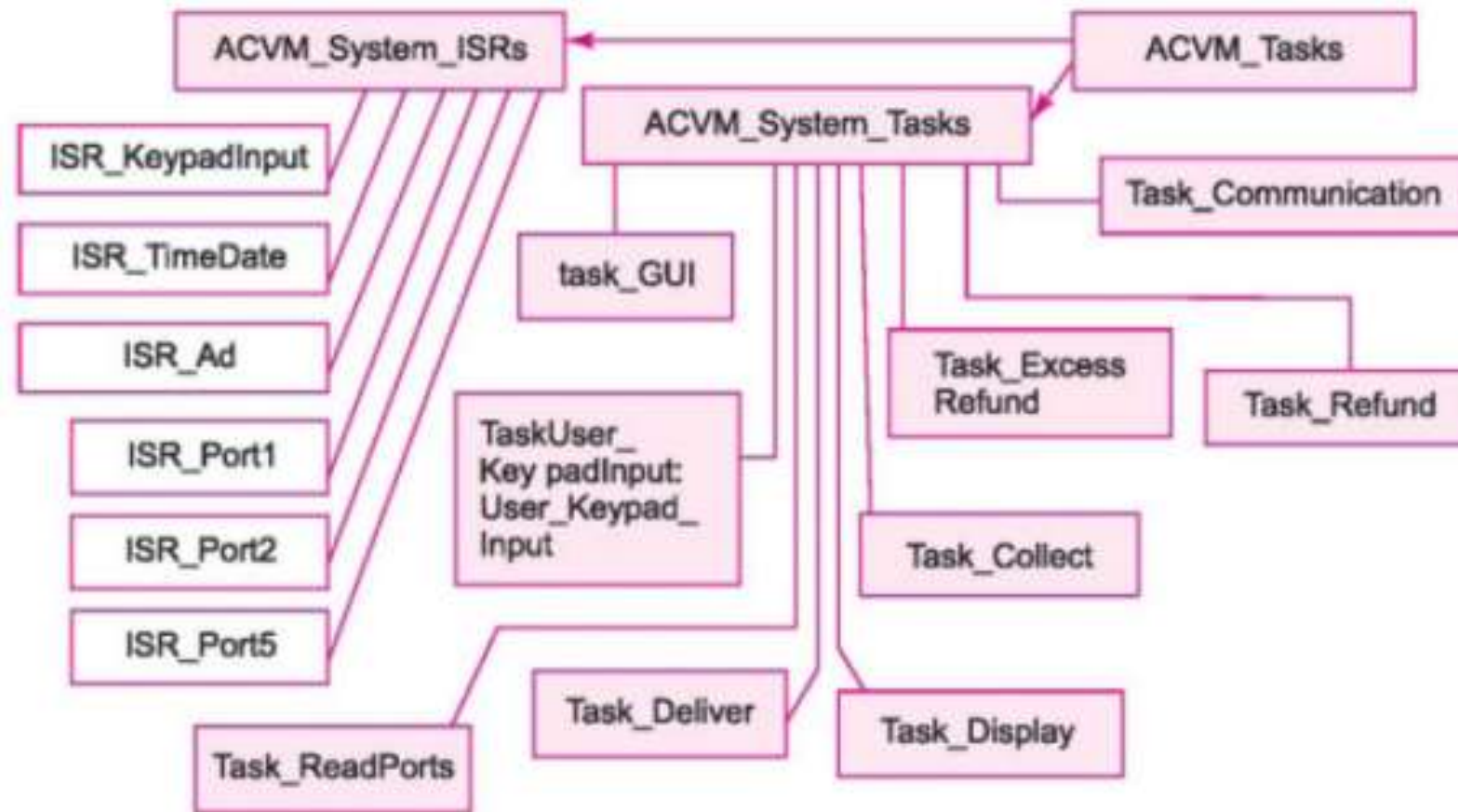
AUTOMATIC CHOCOLATE VENDING MACHINE

ARCHITECTURE – HARDWARE DIAGRAM



AUTOMATIC CHOCOLATE VENDING MACHINE

ARCHITECTURE – SOFTWARE DIAGRAM



AUTOMATIC CHOCOLATE VENDING MACHINE

COMPONENTS - HARDWARE

- ACVM specific hardware to sort the coins of different denomination and each denomination.
- Main Power supply 220 V 50 Hz or 110 V 60 Hz. Internal circuits drive by supply of 5 V 50mA for electronic and 12 V 2 A for mechanical systems.
- A TCP/IP port.
- A 1s resolution timer is obtained by programming 8051 timer.
- Flash memory part of ROM and RAM for storing temporary variables and stack.
- Microcontroller 8051 MX and 8 MB ROM for application codes.

AUTOMATIC CHOCOLATE VENDING MACHINE

COMPONENTS - SOFTWARE

- **Task_ReadPorts:** Waits for the coins and action as per coins collected.
- **Task_Collect:** Waits for coins = or > cost till timeout and act accordingly.
- **Task_Deliver:** Waits for Task_Readports, delivers chocolate, and decreases coins' amount after delivery.
- **Task_Refund:** Waits for refund event and calculate refund amount.
- **Task_Excess Refund:** Refunds the Excess amount
- **Task_Display:** Waits for the message and display as per message.

AUTOMATIC CHOCOLATE VENDING MACHINE

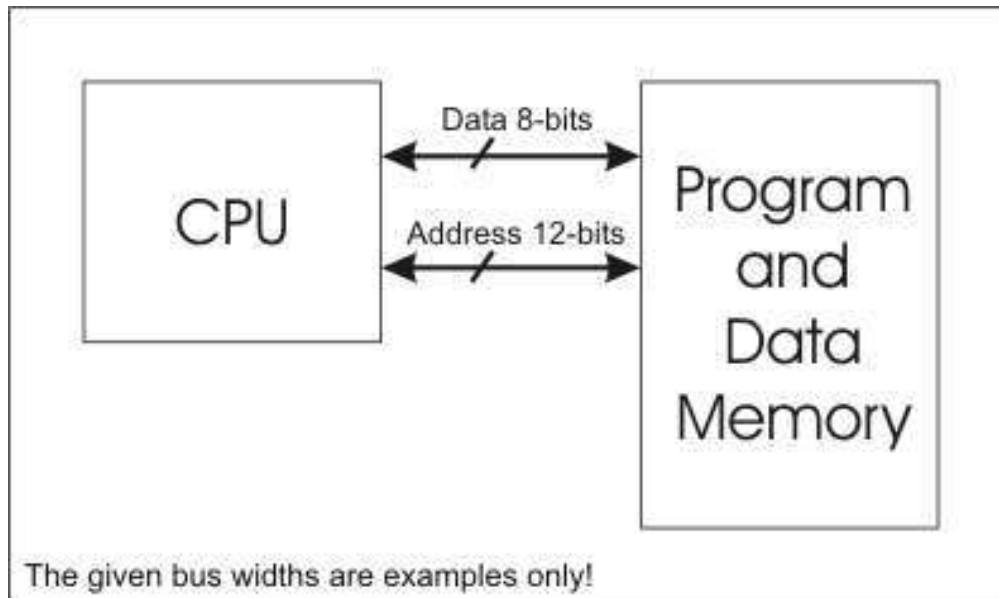
SYSTEM INTEGRATION

- **Test and validation conditions:**
 - All user commands must function correctly.
 - All graphic displays and menus should appear as per the program.
 - Each task should be tested with test inputs.
 - Tested for 60 users per hour.

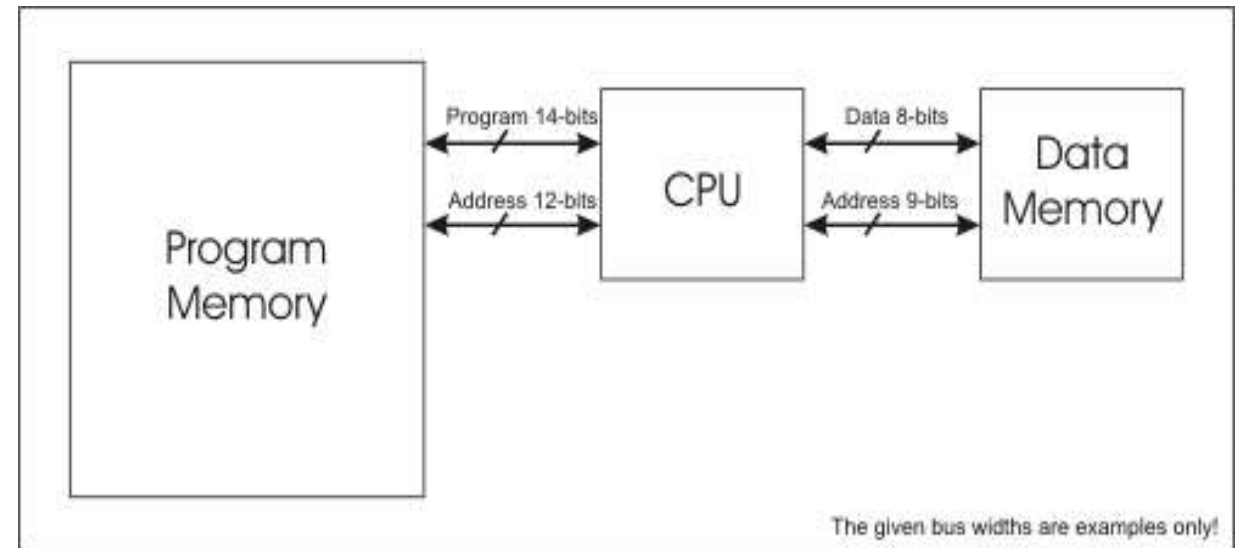
MICROPROCESSOR vs MICROCONTROLLER

Parameters	μP	μC
Architecture	Von-Neumann and RISC architecture	Harvard and CISC architectures.
Flexibility	designer can decide on the amount of ROM, RAM and I/O ports.	fixed amount of on-chip ROM, RAM, I/O ports
Processing Power	High	Low
Design Time	More	Less
Power	High	Low
Protection	Less	More
Cost	High	Low
Examples	8085, 8086	8051

ARCHITECTURE



Von Neumann Architecture



Harvard Architecture

ARCHITECTURE

CISC	RISC
Emphasis on hardware	Emphasis on software
Multiple instruction sizes and formats	Instructions of same set with few formats
Less registers	Uses more registers
More addressing modes	Fewer addressing modes
Extensive use of microprogramming	Complexity in compiler
Instructions take a varying amount of cycle time	Instructions take one cycle time
Pipelining is difficult	Pipelining is easy

X86 Architecture

ARM, Power PC

INTRODUCTION TO INTEL MCS 51 μ C

INTEL MCS 51 MICROCONTROLLER

- ❑ The Intel MCS-51 (commonly termed 8051) is a Harvard architecture, complex instruction set computing (CISC) Architecture, single chip microcontroller (μC) series developed by Intel in 1980 for use in embedded systems
- ❑ Intel's original MCS-51 family was developed using N-type metal-oxide-semiconductor (NMOS) technology
- ❑ But later versions, identified by a letter C in their name (e.g., 80C51) used complementary metal-oxide-semiconductor (CMOS) technology and consume less power

INTEL MCS 51 MICROCONTROLLER

FEATURES

- 8-bit CPU
- 64K bytes on-chip program memory (ROM)
- 128 bytes on-chip data memory (RAM)
- 32 I/O pins arranged as four 8-bit ports (P0 - P3)
- 32 general purpose registers each of 8-bit
- Special Function Registers (SFRs) of 128 bytes
- 16-bit Program Counter
- 8-bit Processor Status Word (PSW) & Stack Pointer
- Two 16-bit timer/counters : T0 and T1
- Two external and three internal vectored interrupts
- One full duplex serial I/O (UART)

INTEL MCS 51 MICROCONTROLLER

8051 FAMILY SERIES

Feature	8051	8052	8031
ROM(bytes)	4K	8K	0K
RAM(bytes)	128	256	128
Timers	2	3	2
I/O pins	32	32	32
Serial port	1	1	1
Interrupt sources	6	8	6

INTEL MCS 51 MICROCONTROLLER

8051 FAMILY SERIES

- ❑ Although the 8051 is the most popular member of the 8051 family, you will not see "8051" in the part number.
- ❑ This is because the 8051 is available in different memory types, such as UV-EPROM, flash, and NV-RAM, all of which have different part numbers.
- ❑ The UV-EPROM version of the 8051 is the 8751. The flash ROM version is marketed by many companies including Atmel Corp. and Dallas Semiconductor.

INTEL MCS 51 MICROCONTROLLER

8051 FAMILY SERIES

- ❑ The Atmel Flash 8051 is called AT89C51, while Dallas Semiconductor calls theirs DS89C4x0 (DS89C420/430/440).
- ❑ The NV-RAM version of the 8051 made by Dallas Semiconductor is called DS5000.
- ❑ There is also an OTP (one-time programmable) version of the 8051 made by various manufacturers.

INTEL MCS 51 MICROCONTROLLER

8051 MAJOR MANUFACTURERS

- ATMEL
- ANALOG DEVICES
- ST MICROELECTRONICS
- DALLAS
- MAXIM
- SILICON LABS
- TEXAS INSTRUMENTS
- MICROCHIP
- ZILOG

INTEL MCS 51 MICROCONTROLLER

8051 ATMEL SERIES

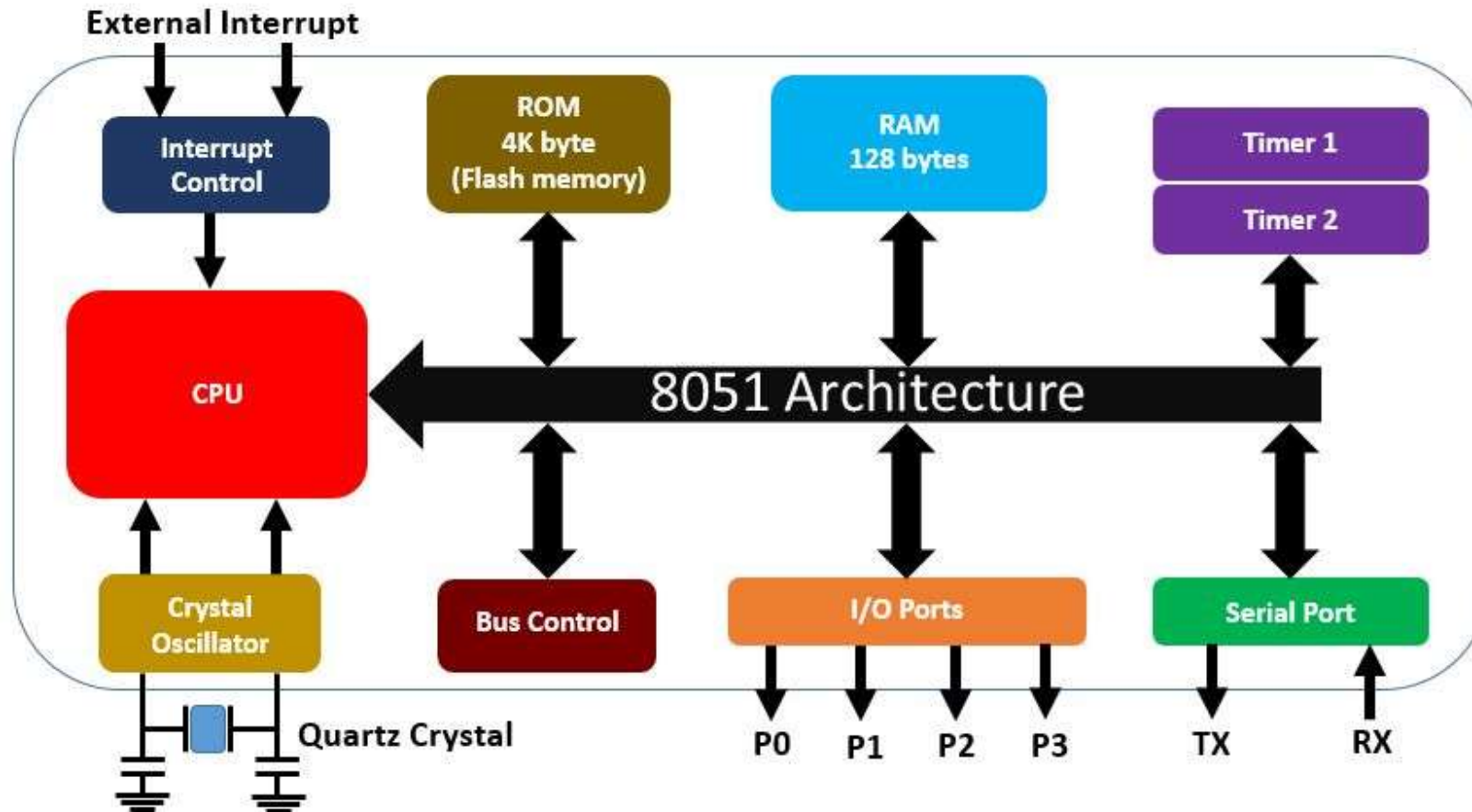
Part Number	ROM	RAM	I/O pins	Timer	Interrupt	Vcc	Packaging
AT89C51	4K	128	32	2	6	5V	40
AT89C52	8K	256	32	3	8	5V	40
AT89C1051	1K	64	15	1	3	3V	20
AT89C2051	2K	128	32	3	8	3V	20
AT89LV51	4K	128	32	2	6	3V	40
AT89LV52	8K	128	32	3	8	3V	40

8051

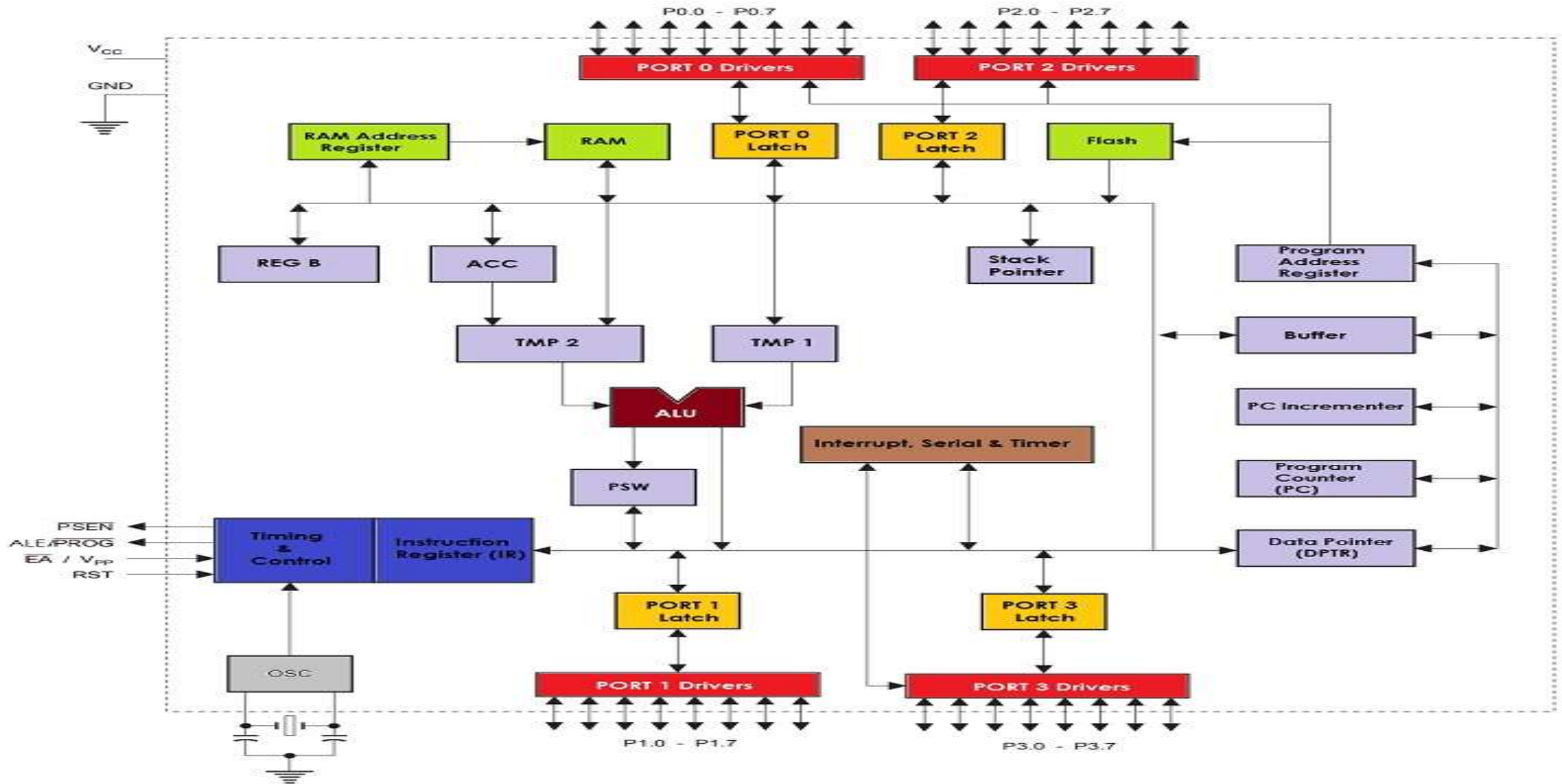
**MICROCONTROLLER
ARCHITECTURE**

8051 MICROCONTROLLER ARCHITECTURE

8051 ARCHITECTURE (Simplified)



8051 MICROCONTROLLER ARCHITECTURE



8051 MICROCONTROLLER ARCHITECTURE

CENTRAL PROCESSING UNIT (CPU)

- ❑ Central Processor Unit (**CPU**) is the brain of any processing device of the microcontroller.
- ❑ It monitors and controls all operations that are performed on the Microcontroller units.
- ❑ The User has no control over the work of the CPU directly.
- ❑ It reads program written in ROM memory and executes them and do the expected task of that application.

8051 MICROCONTROLLER ARCHITECTURE

INTERRUPT CONTROL

- ❑ There are five interrupt sources in 8051 Microcontroller and **interrupt control** section control all these interrupts.
- ❑ Two external interrupts (INT0 & INT1), two timer (TF0 & TF1) interrupts and one serial port (RI / TI) interrupt.
- ❑ The Microcontroller 8051 can be configured in such a way that it temporarily terminates or pause the main program at the occurrence of interrupt. When subroutine is completed then the execution of main program starts as usual.

8051 MICROCONTROLLER ARCHITECTURE

RAM & ROM

- ❑ Microcontroller 8051 has 4K of Code Memory or Program memory that is it has 4KB ROM and RAM of 128 bytes.
- ❑ The memory which is used to store the program of Microcontroller, is known as code memory or Program memory . It is known as 'ROM'(Read Only Memory).
- ❑ Microcontroller also requires a memory to store data or operands temporarily. This memory is known as Data Memory and we use 'RAM'(Random Access Memory) for this purpose.

8051 MICROCONTROLLER ARCHITECTURE

BUS CONTROL

- ❑ **Bus control** section of 8051 is responsible for controlling the operation of address and data bus.
- ❑ **Bus:** Basically Bus is a collection of wires which work as a communication channel or medium for transfer of Data.
- ❑ **Buses are of two types:**
 - **Address Bus:** Microcontroller 8051 has a 16 bit address bus. It used to address memory locations.
 - **Data Bus:** Microcontroller 8051 has 8 bits data bus. It is used to carry data.

8051 MICROCONTROLLER ARCHITECTURE

CRYSTAL OSCILLATOR

- ❑ **Crystal Oscillator:** Since Microcontroller is a digital circuit device, therefore it requires clock for its operation.
- ❑ For this purpose, Microcontroller 8051 has oscillator circuitry section which works as a clock source for Central Processing Unit.
- ❑ As the output pulses of oscillator are stable therefore it enables synchronized work of all parts of 8051 Microcontroller.

8051 MICROCONTROLLER ARCHITECTURE

I/O PORTS

- ❑ **I/O Ports:** To connect any external devices or peripherals we require I/O interfacing ports in the microcontroller.
- ❑ All 8051 microcontrollers have 4 I/O ports each comprising 8 bits which can be configured as input (1) or an output (0), depends on its logic state.
- ❑ Accordingly, in total of 32 input/output pins enabling the microcontroller to be connected to peripheral devices are available for use.

8051 MICROCONTROLLER ARCHITECTURE

TIMERS/COUNTERS

- ❑ **Timers/Counters:** 8051 Microcontroller 8051 two 16-bit timers and counters: Timer 0 and Timer 1.
- ❑ They can be used either as timers to generate a time delay or as counters to count events happening outside the microcontroller.
- ❑ Since the 8051 has an 8-bit architecture, each 16-bit is accessed as two separate registers of low byte and high byte.

8051 MICROCONTROLLER ARCHITECTURE

SERIAL PORT

- ❑ **Serial port:** The 8051 contains one Serial port or UART (Universal Asynchronous Receiver Transmitter)
- ❑ The serial port is full-duplex so, it can transmit and receive simultaneously
- ❑ Two port pins are used to provide the serial interface P3.0 is the receive pin (RXD) P3.1 is the transmit pin (TXD)
- ❑ This serial port that can be programmed to operate in one of four different modes and at a range of frequencies.

INTRODUCTION TO ARM7

ABOUT ARM

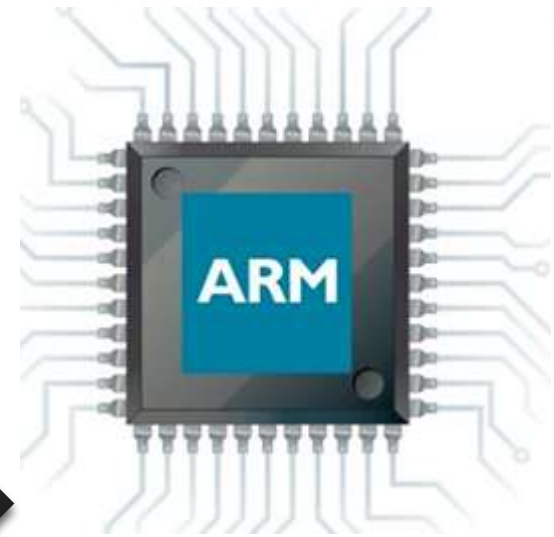
ARM was founded as **A**dvanced **R**ISC **M**achines in 1990

ARM is the world's leading provider of RISC (Reduced Instruction Set Computer) based microprocessor

ARM does not manufacture silicon by itself, instead it license ARM core design to many semiconductor partners around the world.

Used especially in portable devices due to low power consumption and reasonable performance (MIPS/watt)

ARM also develop technologies to assist designing of ARM architecture which includes, software tools, boards, debug hardware, application software, bus architectures, peripherals etc.,



ARM PROCESSOR - FEATURES

32/64-bit RISC-processor core

Load and Store type architecture

Fast interrupt response

uniform and fixed length instruction set (32-bit ARM & 16-bit Thumb)

small core size and power-efficiency

Multiple stage Pipelined operation

wide range of clock frequency ranging from 1MHz to few GHz

High performance and High code density

8 / 16 / 32 / 64 -bit data types

Various modes of operation (USR, SYS, FIQ, IRQ, SVC, ABT, UND)

Good speed / power consumption ratio

HIGH PERFORMANCE, LOWER COST, LESS POWER

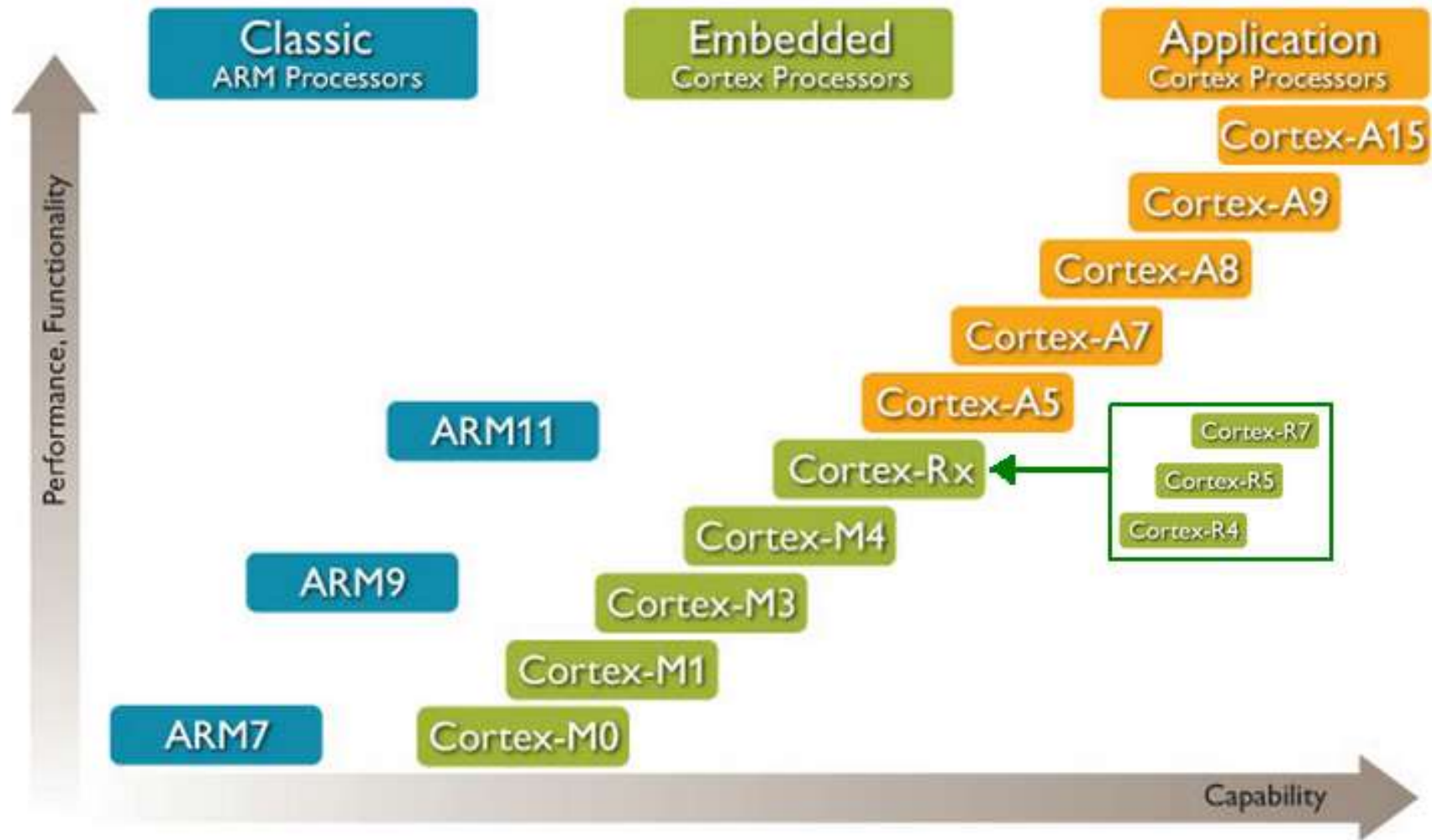
ARM PROCESSOR – APPLICATION DOMAINS

- ❖ Home Appliances
- ❖ Consumer Electronics
- ❖ Home Automation / Security
- ❖ Computer components
- ❖ Automotive
- ❖ Health care
- ❖ Military
- ❖ Industrial
- ❖ Business
- ❖ Retail
- ❖ Communications
- ❖ Avionics
- ❖ Banking

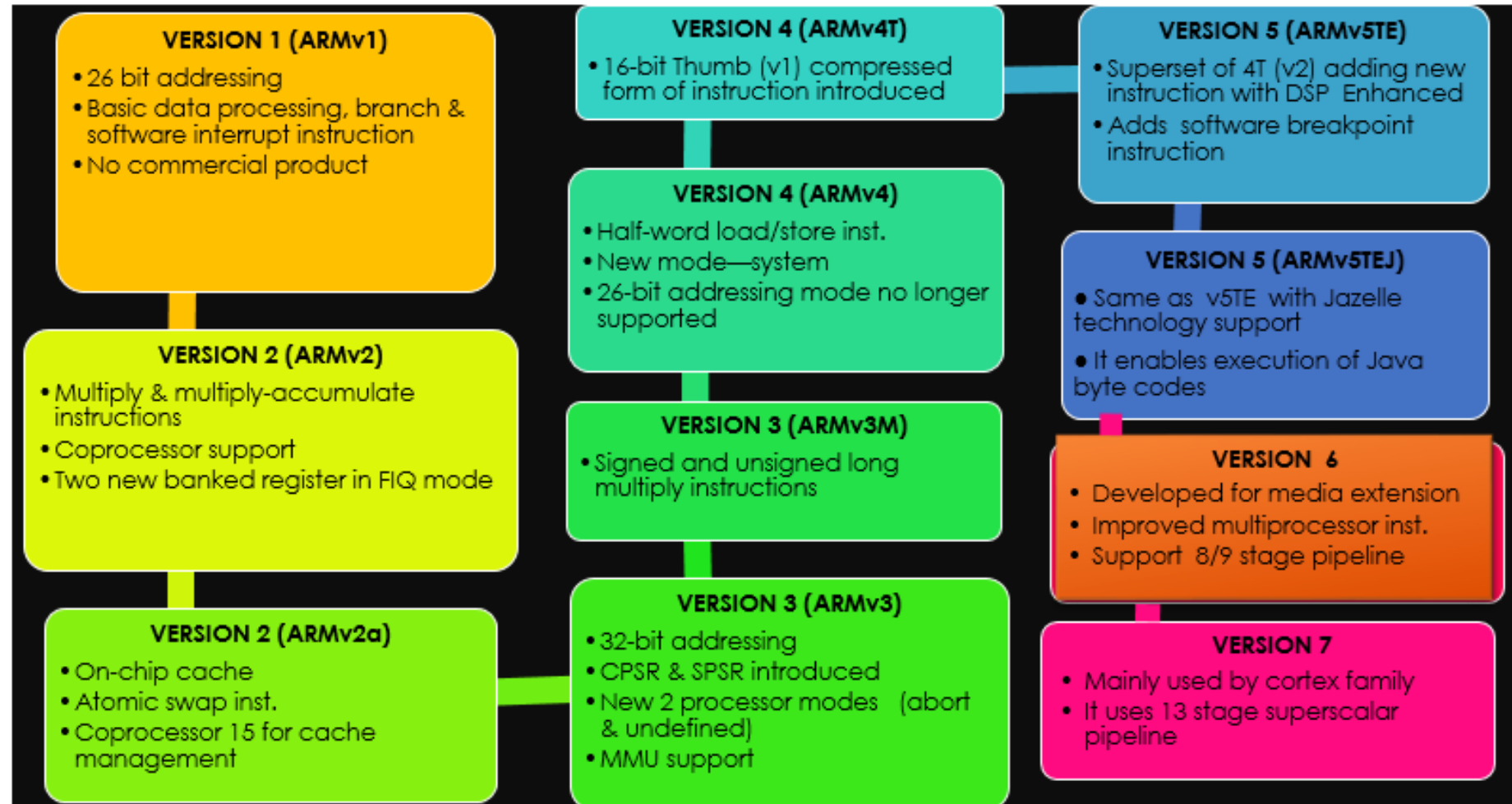
ARM ARCHITECTURE - FEATURES

- ARM. a large set of registers
- ARM. a load-store architecture
- ARM. 3-address instructions
- ARM. Single-cycle execution
- ARM. conditional execution of every instruction
- ARM. powerful load and store multiple register instructions
- ARM. Simple addressing mode
- ARM. extension through the coprocessor instruction set

ARM PROCESSOR - FAMILIES



ARM ARCHITECTURE - VERSIONS



ARM ARCHITECTURE - CURRENT VERSION

Application Profile ARMv8-A

- 32-bit and 64-bit
- A32, T32 and A64 instruction sets
- Virtual memory system
- Supporting rich operating systems



Real-time Profile ARMv8-R

- 32-bit
- A32 and T32 instruction sets
- Protected memory system
- Optimized for real-time systems



Microcontroller Profile ARMv8-M

- 32-bit
- T32 / Thumb® instruction set only
- Protected memory system
- Optimized for microcontroller applications



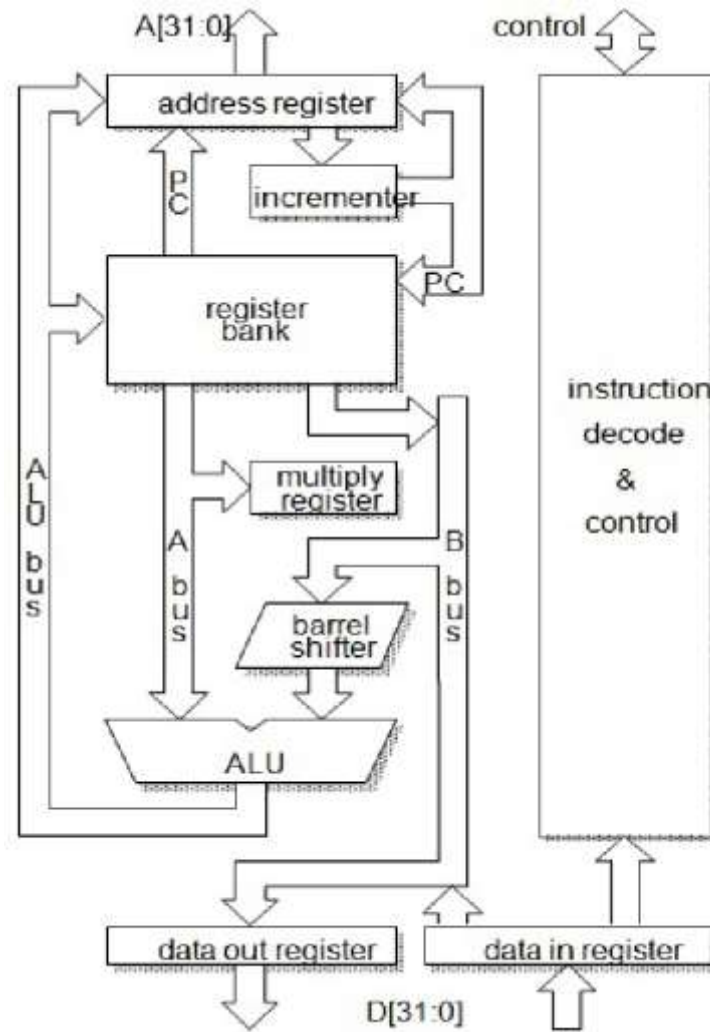
ARM ARCHITECTURE - EVOLUTION

Core	Architecture
ARM1	v1
ARM2	v2
ARM2as, ARM3	v2a
ARM6, ARM600, ARM610	v3
ARM7, ARM700, ARM710	v3
ARM7TDMI, ARM710T, ARM720T, ARM740T	v4T
StrongARM, ARM8, ARM810	v4
ARM9TDMI, ARM920T, ARM940T	V4T
ARM9E-S, ARM10TDMI, ARM1020E	v5TE
ARM10TDMI, ARM1020E	v5TE
ARM11 MPCore, ARM1136J(F)-S, ARM1176JZ(F)-S	v6
Cortex-A/R/M	v7

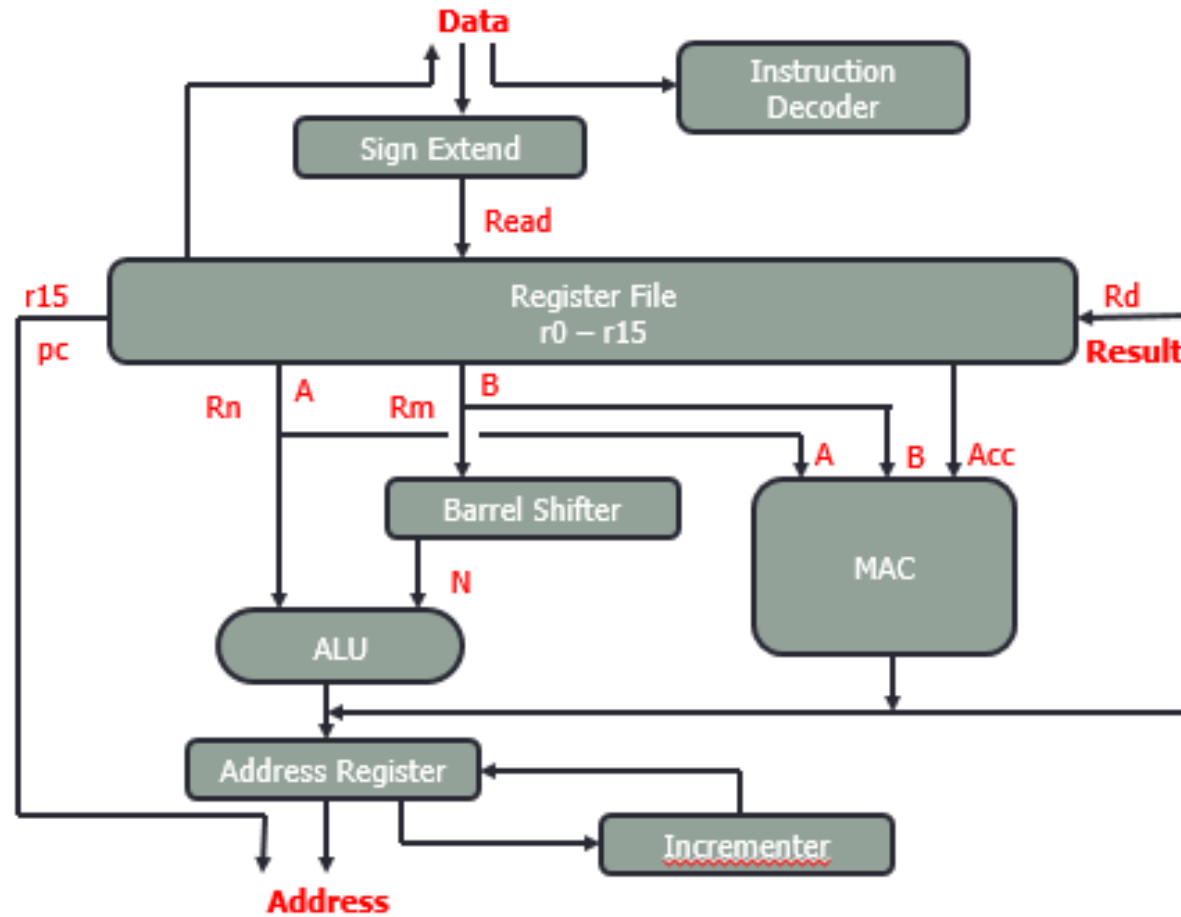
• ARM{x}{y}{z}{T}{D}{M}{I}{E}{J}{F}{S}

- x : Family
- y : MMU/Protection unit
- z : Cache
- T : Thumb 16-bit decoder
- D : JTAG Debug
- M : Fast multiplier
- I : Embedded ICE macrocell
- E : DSP Enhanced
- J : Jazelle Technology
- F : Vector floating point unit
- S : Synthesizable version

ARM7 BASIC ARCHITECTURE



ARM CORE DATAFLOW MODEL



ARM CORE DATAFLOW MODEL

☐ Instruction Decoder:

- ☐ The instruction decoder translates instructions before they are executed.
- ☐ The ARM processor, like all RISC processors, uses a load-store architecture.
- ☐ Load instructions copy data from memory to registers in the core, and conversely the store

☐ Register File

- ☐ A storage bank made up of 32-bit registers.
- ☐ Most instructions treat the registers as holding signed or unsigned 32-bit values

ARM CORE DATAFLOW MODEL

- ❑ **Sign extend hardware**
 - ❑ Converts signed 8-bit and 16-bit numbers to 32-bit values
- ❑ **ALU & MAC:**
 - ❑ Takes the register values R_n and R_m from the A and B buses and computes a result.
 - ❑ Data processing instructions write the result in R_d directly to the register file.
 - ❑ Load and store instructions use the ALU to generate an address to be held in the address register and broadcast on the Address bus.

ARM CORE DATAFLOW MODEL

- ❑ **Source Registers:**
 - ❑ ARM instructions have two source registers, Rn & Rm, and a single result or destination register, Rd.
 - ❑ Source operands are read from the register file using the internal buses A and B, respectively.
- ❑ **Address Register & Incrementer**
 - ❑ For load and store instructions, the incrementer updates the address register before the core reads or writes the next register value from or to the next sequential memory location.
 - ❑ The processor continues executing instructions until an exception or interrupt changes the normal execution flow.

ARM CORE DATAFLOW MODEL

- ❑ **Barrel Shifter:**
 - ❑ register Rm alternatively can be preprocessed in the barrel shifter before it enters the ALU.
 - ❑ Provides five types of shift
 - ❑ Logic Shift left (LSL)
 - ❑ Logic Shift Right (LSR)
 - ❑ Arithmetic Shift Right (ASR)
 - ❑ Rotate Right(ROR)
 - ❑ Rotate Right Extended (RRX)
- ❑ After passing through the functional units, the result in Rd is written back to the register file using the Result bus.

ARM PROCESSOR - MODES

- ❑ The processor mode determines which registers are active and the access rights to the cpsr register itself.
- ❑ Each processor mode is either privileged or nonprivileged:
 - A PRIVILEGED mode allows full read-write access to the CPSR
 - A NONPRIVILEGED mode only allows read access to the control field in the CPDR but still allows read-write access to the condition flags.
- ❑ There are seven processor modes in total: six privileged modes (abort, fast interrupt request, interrupt request, supervisor, system, and undefined) and one nonprivileged mode (user).

ARM PROCESSOR - MODES

Exception modes

Mode	Description	
Supervisor (SVC)	Entered on reset and when a Supervisor call instruction (SVC) is executed	Privileged modes
FIQ	Entered when a high priority (fast) interrupt is raised	
IRQ	Entered when a normal priority interrupt is raised	
Abort	Used to handle memory access violations	
<u>Undef</u>	Used to handle undefined instructions	
System	Privileged mode using the same registers as User mode	Unprivileged mode
User	Mode under which most Applications / OS tasks run	

ARM PROCESSOR – REGISTER SET

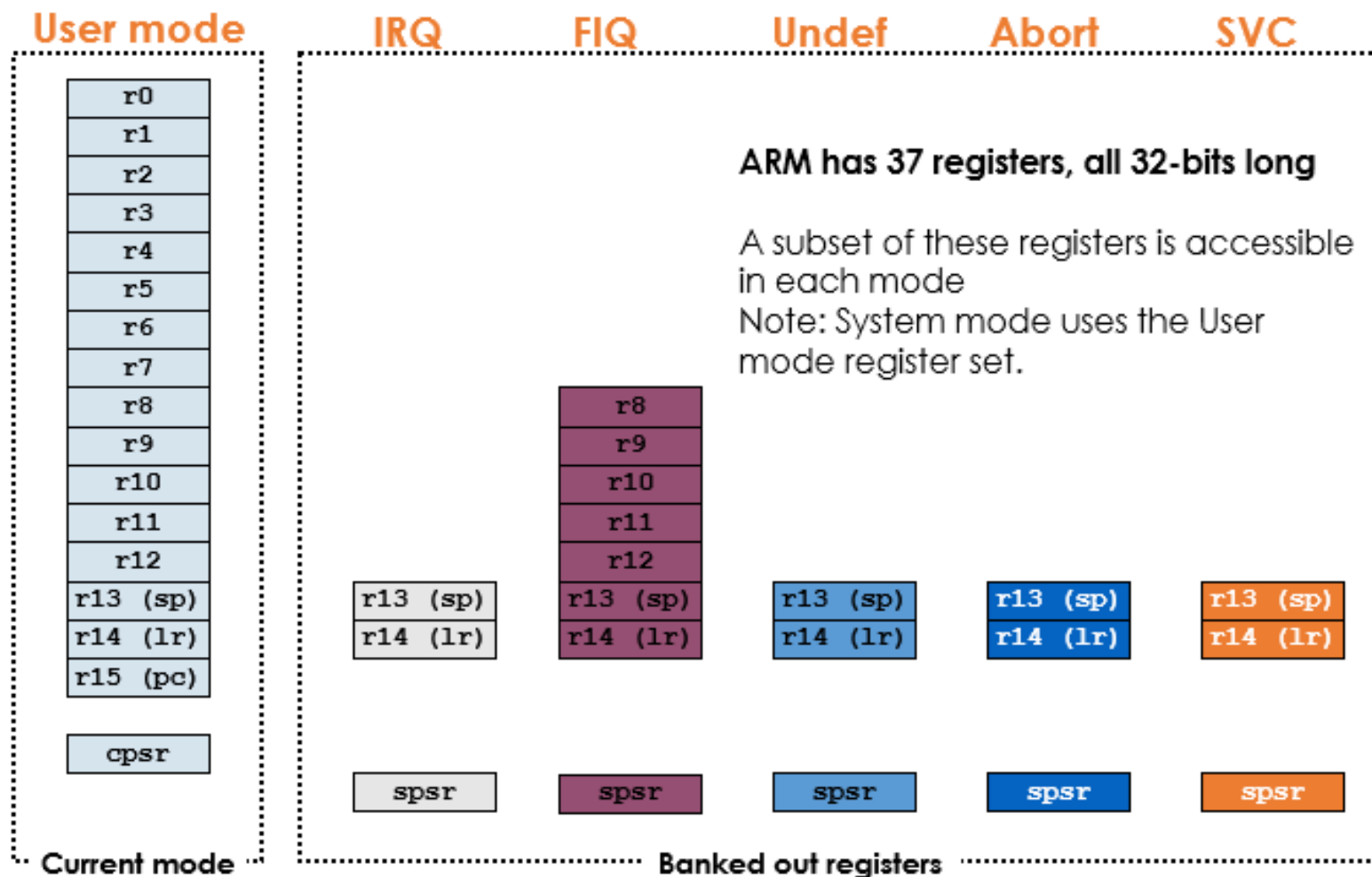
UNBANKED REGISTER

This means that each of them refer to the same 32-bit physical register in all processor modes. They are completely general purpose registers, and can be used whenever an instruction allows GPR to be specified

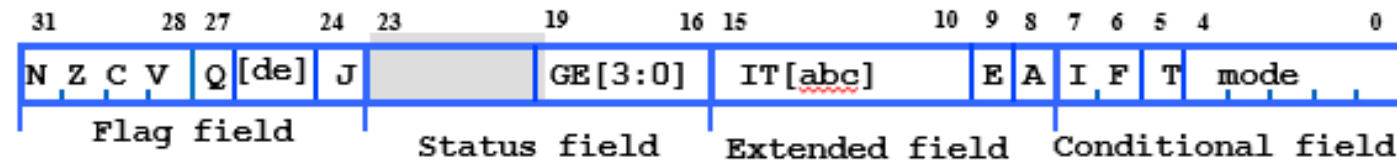
BANKED REGISTER

They are available only when the processor is in a particular mode. Where particular register is intended, without depending on the current processing mode a more specific name is used.

ARM PROCESSOR – REGISTER SET



ARM PROCESSOR – CPSR REGISTER



- **Condition code flags**
 - N = Negative result from ALU
 - Z = Zero result from ALU
 - C = ALU operation Carried out
 - V = ALU operation oVerflowed
- **Sticky Overflow flag - Q flag**
 - Indicates if saturation has occurred
- **SIMD Condition code bits – GE[3:0]**
 - Used by some SIMD instructions
- **IF THEN status bits – IT[abcde]**
 - Controls conditional execution of Thumb instructions
- **T bit**
 - T = 0: Processor in ARM state
 - T = 1: Processor in Thumb state
- **J bit**
 - J = 1: Processor in Jazelle state
- **Mode bits**
 - Specify the processor mode
- **Interrupt Disable bits**
 - I = 1: Disables IRQ
 - F = 1: Disables FIQ
- **E bit**
 - E = 0: Data load/store is little endian
 - E = 1: Data load/store is bigendian
- **A bit**
 - A = 1: Disable imprecise data aborts

Mode bits	
M[4:0]	Mode
0b10000	User
0b11111	System
0b10001	FIQ
0b10010	IRQ
0b10011	Supervisor
0b10111	Abort
0b11011	Undefined

PIC

FEATURES

- ❑ This powerful (200 nanosecond instruction execution) yet easy-to-program (RISC instructions).
- ❑ CMOS FLASH-based 8-bit microcontroller packs Microchip's powerful PIC® architecture into an 40-pin dual in-line (DIP)
- ❑ The PIC16F877A features 256 bytes of EEPROM data memory, self programming, an ICD, 2 Comparators, 8 channels of 10-bit Analog-to-Digital (A/D) converter, 2 input capture/output compare/PWM functions, the synchronous serial port can be configured as either 3-wire Serial Peripheral Interface (SPI™) or the 2-wire Inter-Integrated Circuit (I²C™) bus and a Universal Synchronous Asynchronous Receiver Transmitter (USART).

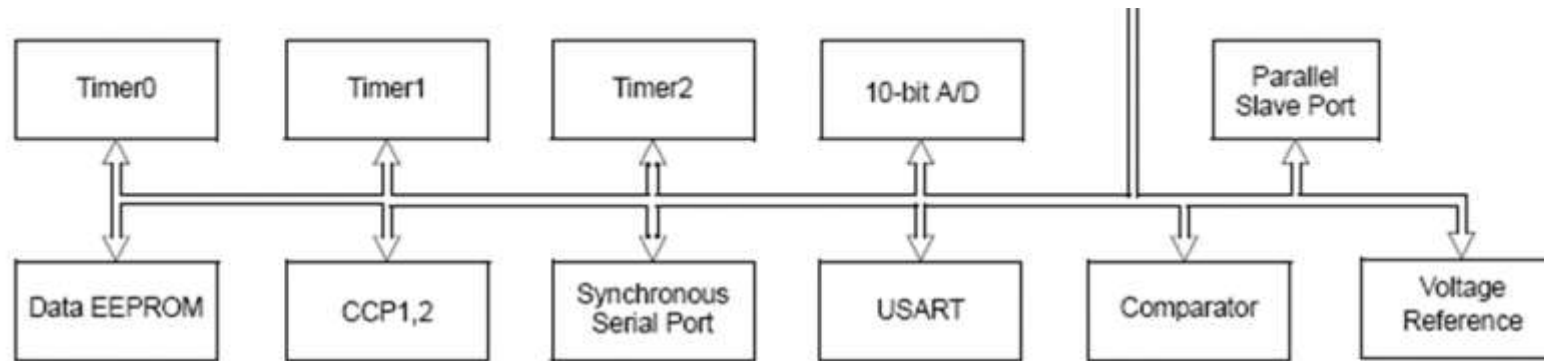
PIC

MEMORY

- ❑ 8 K of flash program ROM (14-bits wide).
- ❑ Each of the 35 RISC instructions fits in just one 14-bit wide location, permitting fast (200 ns) instruction execution time, if microcontroller is clocked at its maximum allowable clock rate of 20 MHz.
- ❑ 368 bytes of volatile data RAM “File Register” locations (8-bits wide) for temporary storage of data.
- ❑ 256 bytes of nonvolatile EEPROM for storage of calibration data, user-customized option settings, etc.

PIC

PIC16F877A HARDWARE BLOCK DIAGRAM



Device	Program Flash	Data Memory	Data EEPROM
PIC16F874A	4K words	192 Bytes	128 Bytes
PIC16F877A	8K words	368 Bytes	256 Bytes

PIC

Special Functions of PIC16F877A I/O Pins

- ❑ Three hardware timers: Timer 0 (RA4/T0CK1 pin), Timer 1 (RC0/T1OSO/TICK1 and RC1/T1OSI pins), and Timer 2 (no external pins are associated with this timer – this timer can be set to periodically interrupt via the PR2 period register).
- ❑ 10-bit, 8-channel analog-to-digital converter (analog inputs on Ports A and E)
- ❑ 8-bit parallel slave port (PSP) for making the PIC look like an addressable 8-bit peripheral register on a host computer's bus (Port D for the 8-bit data bus connection)
- ❑ PORT E for the RD\, WR\, and CS\ bus control lines).

PIC

Special Functions of PIC16F877A I/O Pins

- ❑ Input Capture – Output Compare – Pulse Width Modulation pins (RC2/CCP1 and RC1/T1OSI/CCP2).
- ❑ Synchronous serial data port (SDI, SDO, and SCK pins) used for implementing master and slave serial peripheral interface SPI and I2C serial interfaces.
- ❑ Asynchronous serial data port, or Universal Asynchronous Receiver-Transmitter, (RC6/TX and RC7/RX pins).
- ❑ Two analog comparators with programmable analog reference voltage (PORT A pins are used for the analog comparator inputs and also the digital comparator outputs).

PIC

	8051	PIC	AVR	ARM
Bus width	8-bit for standard core	8/16/32-bit	8/32-bit	32-bit mostly also available in 64-bit
Communication Protocols	UART, USART, SPI, I2C	PIC, UART, USART, LIN, CAN, Ethernet, SPI, I2S	UART, USART, SPI, I2C, (special purpose AVR support CAN, USB, Ethernet)	UART, USART, LIN, I2C, SPI, CAN, USB, Ethernet, I2S, DSP, SAI (serial audio interface), IrDA
Speed	12 Clock/instruction cycle	4 Clock/instruction cycle	1 clock/ instruction cycle	1 clock/ instruction cycle
Memory	ROM, SRAM, FLASH	SRAM, FLASH	Flash, SRAM, EEPROM	Flash, SDRAM, EEPROM
ISA	CISC	Some feature of RISC	RISC	RISC
Memory Architecture	Von Neumann architecture	Harvard architecture	Modified	Modified Harvard architecture
Power Consumption	Average	Low	Low	Low

PIC

	8051	PIC	AVR	ARM
Families	8051 variants	PIC16,PIC17, PIC18, PIC24, PIC32	Tiny, Atmega, Xmega, special purpose AVR	ARMv4,5,6,7 and series
Community	Vast	Very Good	Very Good	Vast
Manufacturer	NXP, Atmel, Silicon Labs, Dallas, Cyprus, Infineon, etc.	Microchip Average	Atmel	Apple, Nvidia, Qualcomm, Samsung Electronics, and TI etc.
Cost (as compared to features provide)	Very Low	Average	Average	Low
Other Feature	Known for its Standard	Cheap	Cheap, effective	High speed operation
Popular Microcontroller s	AT89C51, P89v51, etc.	PIC18fXX8, PIC16f88X, PIC32MXX	Atmega8, 16, 32, Arduino Community	LPC2148, ARM Cortex-M0 to ARM Cortex-M7, etc.